

CXU Playground

RISC-V Custom Extension Development Framework

Petro Mozil

Ukrainian Catholic University

2026

This Work's Goals

CFU — exists

Reusable across cores

But: one extension per thread ·
assembly-only · no shared state

CXU — spec only

Multiple extensions · shared
state · C-level ABI

Gap: no hardware · no sandbox

CXU Playground — this work

Hardware implementation + full
open-source sandbox running real
Linux

System stack

User-space benchmarks AES · GZIP · TFLite Micro

CXU runtime library header-only C API · cxidx multiplexing · no inline assembly

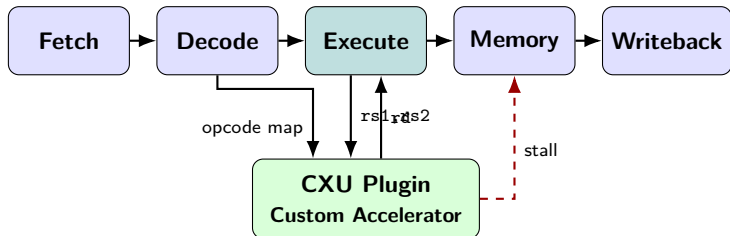
Linux (Buildroot) OpenSBI → U-Boot → kernel · MMU · perf profiling

LiteX SoC auto-generated · Wishbone bus · UART / Ethernet / DDR3

VexiiRiscv + CXU Plugin SpinalHDL · execute-stage accelerator · cxidx/cxdata CSRs

Hardware: Digilent Arty S7-50 · Spartan-7 XC7S50 · 50 MHz · 256 MB DDR3

CXU plugin in the VexiiRiscv pipeline



What the plugin does

- ▶ Intercepts custom-0/1/2/3 opcodes at Decode
- ▶ Forwards rs1/rs2 directly from register file
- ▶ Stalls pipeline for multi-cycle operations
- ▶ cxidx CSR selects the active extension

Why it matters

- ▶ Zero-copy operand path — no DMA, no bus transaction
- ▶ Single-cycle or stalled — no polling loop in software
- ▶ Multiple extensions share the same hardware slot
- ▶ C callable — runtime library hides the CSR writes

The Playground environment

SoC generation

One Python script builds the entire SoC:

- ▶ VexiiRiscv with CXU plugin wired in
- ▶ Memory map, interrupt controller, DTS auto-generated
- ▶ Bitstream → FPGA in a single make call

Linux environment

Buildroot cross-compiles:

- ▶ Linux kernel + root filesystem
- ▶ OpenSBI → U-Boot boot chain
- ▶ perf + minstret for cycle-accurate profiling
- ▶ CXU accessible from ordinary user-space C

Runtime library

Header-only C API:

- ▶ No inline assembly in application code
- ▶ Manages `cxidx` register transparently
- ▶ Paired software reference for every extension

Developer workflow

Edit SpinalHDL extension → rebuild SoC → boot Linux → run benchmark — all in one repo:
<https://github.com/pmozil/CXU-playground>

Three extensions — one framework

AES GF(2⁸) accelerator

Single-cycle finite-field instruction
replaces a 5-instruction inner loop
per byte

5.25× speedup

stable · 128 B – 16 KB

209 logic cells · consistent with
Marshall et al. (4× RV32, 10×
RV64)

GZIP / DEFLATE

4 combinational instructions:

bitrev · mask · huft_idx ·
copy_addr

~10% CPU time

end-to-end reduction

20 cells · LZ77 search & filesystem
I/O dilute gains; accelerated
primitives: 12–15% of inner loop

TFLite Micro MAC

Fused multiply-accumulate for INT8
quantised inference

~13% latency

1577 ms → 1396 ms

Folded into pipeline (no top-level
instance) · confirms memory-bound
ceiling without RAM access

Even a handful of tightly coupled instructions yields substantial gains for arithmetic-intensive workloads on commodity FPGA hardware running real Linux.

Limitations & future work

Current limitations

- ▶ **CSR latency:** `cxidx/cxdata` traverse the Wishbone bus — full transaction per state access. Design for one-time configuration.
- ▶ **No context-switch support:** CXU state not saved/restored on preemption — safe for single-process only.
- ▶ **No Device Tree entries:** extension addresses are hard-coded in the runtime library.

Future directions

- ▶ Kernel driver: `cxidx/cxdata` save/restore in task struct → multi-process safety
- ▶ Native pipeline CSRs (bypass Wishbone) → zero-cycle state access
- ▶ LiteX DTS emission for capability discovery
- ▶ Post-quantum crypto: NTT butterfly / modular reduction — literature shows up to $9.6\times$ for Kyber

Summary

What was built

- ▶ **First CXU hardware implementation** — SpinalHDL plugin for VexiiRiscv: opcode interception, pipeline stall, CSR support
- ▶ **CXU Playground** — open-source, full-stack sandbox: LiteX SoC + Buildroot Linux + C runtime library
- ▶ **Three validated extensions** — AES GF(2^8) · GZIP DEFLATE · TFLite Micro MAC

Key takeaway

A composable custom extension interface previously existing only as a design document now runs on real hardware under Linux — and the playground makes it reproducible.

<https://github.com/pmozil/CXU-playground>